

Team 5: Final Project Report

DD2358 - Introduction to High Performance Computing

Bhatta Mridula Chandrika
 Tim Luca Eberle
 Akira-Miranda Adeyomi Adeniran-Lowe

September 27, 2026

Grade Expectations:

Bhatta Mridula Chandrika: A

Tim Luca Eberle: B

Akira-Miranda Adeyomi Adeniran-Lowe: B

1 Introduction

The Visualisation of 3D scalar fields is one of the most computationally expensive visualisation techniques used in fields such as medical imaging, scientific visualisation, and computational modelling. This project focuses on the performance analysis and optimisation of a volume ray casting algorithm, utilising a Python-based volume renderer to map volumetric data onto a 2D image plane. Due to high computational and memory bandwidth requirements, volume rendering serves as an ideal case study for analysing high-performance computing systems.

1.1 Problem Background

Rendering a volume entails the rendering of rays to a 3D volume, which entails a lot of sampling and compositing operations for each pixel in the output image. Even as the Python code [1, 2] is a functional implementation for rendering volumes, the nature of the pixel calculation is such that it is bound to pose a number of performance issues. This project seeks to examine the nature of the code in a Python environment to determine how efficiency can be improved.

Mathematically, volume rendering can be described as the numerical evaluation of the volume rendering integral along each viewing ray. For every pixel of the output image, a ray is cast through the volumetric dataset and sampled at discrete intervals. At each sample position, the scalar data value is mapped to colour and opacity using a transfer function. The final pixel colour is then computed through front-to-back alpha compositing, where the accumulated colour is updated according to the opacity of each sampled voxel. Because this process must be repeated for every pixel and for many samples along each ray, the total number of computations grows rapidly with increasing volume resolution, making efficient implementation crucial for high-resolution datasets.

1.2 Primary objectives and Scope

The fundamental aim of this project is to transform the initial implementation [2] into a high-performance application. This can be achieved by implementing the following objectives:

- **Performance Characterisation:** Performance Characterisation: This project will characterise the computation and memory system performance of the initial Python implementation to get a comprehensive baseline.
- **Identification of Bottlenecks:** This project will identify the areas of the initial Python implementation where performance degradation occurs.
- **Optimisation Implementation:** This project will implement a variety of state-of-the-art optimisation strategies. This will include the implementation of compiler optimisations using Cython, task and process parallelisation using Dask and Multiprocessing, and the use of hardware accelerators using PyTorch on GPUs.
- **Performance Evaluation:** This project will evaluate the performance impact of the implemented optimisations critically. This will include the evaluation of the execution time, speed-up, and scalability of the optimised code compared to the initial Python serial code on a variety of varying 3D dataset sizes.

All code used in this project can be found [here](#).

2 Methodology

2.1 Dataset Selection

To comprehensively characterise the computation and memory system performance of the implementation, a diverse set of 3D volumes was selected from publicly available datasets [3]. These datasets represent various physical and biological structures (e.g. bone, sponge, okinawa forams) and possess varying degrees of sparsity and internal complexity, which directly influences the computational workload.

Rather than evaluation on a single problem size, the datasets were scaled across distinct tiers to stress different levels of the hardware's memory hierarchy:

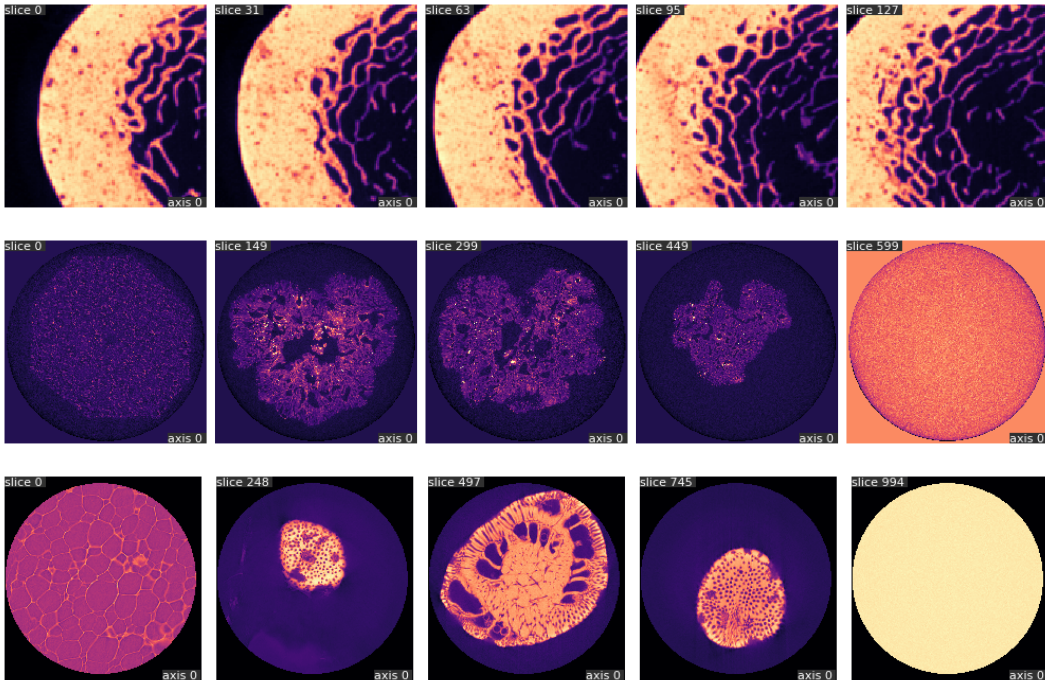


Figure 1: Renders of some of the chosen volumes used in this investigation.

- Small datasets (2 MB - 9 MB) were selected to establish a baseline for computational throughput.
- Medium-sized datasets were chosen to exceed cache capacities, forcing the algorithm to rely on main memory bandwidth.
- Large datasets > 7 GB, were incorporated to severely stress-test system limits.

By evaluating the algorithm across these different scales of datasets, we can quantify exactly what bottlenecks to address and moreover inform the design of our subsequent optimisation techniques.

2.2 Performance Characterisation and Bottleneck Identification

Before starting performance optimization, it is important to identify which parts of the code consume most of the execution time. Optimizing these parts usually provides the greatest improvement. To find them, a technique called profiling is used.

During profiling, all plotting functions are disabled so that they do not influence the measured execution time. As a first step, the “cProfile” profiler is used to determine which functions require the most time during program execution. This profiler measures how often each function is called and how much total time is spent inside the function.

While this approach is well suited for identifying slow functions, it does not show which specific lines inside the function cause the delay. Therefore, a second profiling step is performed using the “line profiler”. This tool measures the execution time of each individual line within the selected function. It allows a more detailed analysis of the functions that were previously identified as performance-critical with the “cProfile” profiler. As a result, it becomes easier to optimize the exact part of the code that causes the performance bottleneck.

Using these two profiling methods makes it possible to identify where the largest improvements in execution time can be achieved. However, performance is related not only to computation time but also to memory usage. For this reason, a memory profiler can also be used. It shows how the memory usage changes during program execution and helps to detect inefficient operations, such as unnecessary copies of large arrays or steadily increasing memory consumption.

2.3 Optimization Implementation

To improve the performance of the baseline volume rendering implementation, several optimization techniques were applied. These optimizations focus on reducing computational overhead and improving parallel execution of the rendering process. Specifically, compiler-level optimization using Cython and parallelization approaches using multiprocessing and Dask were explored. Each method targets different performance bottlenecks within the pipeline.

2.3.1 Compiler Techniques: Cython Implementation

The original volume rendering implementation was written in pure Python. However, Python introduces interpreter overhead, particularly in loops and repeated numerical operations. To reduce this overhead, Cython was used to compile performance-critical sections of the code into optimized C extensions.

The main bottleneck in the rendering pipeline occurs during the compositing stage, where each slice of the interpolated camera grid is processed sequentially. In the baseline implementation, this operation is performed using a Python loop:

```

01:
+02: import numpy as np
03: cimport numpy as np
04: from libc.math cimport exp, log
05:
+06: def render_volume_fast(double[:, :, :] camera_grid):
07:
+08:     cdef int Nx = camera_grid.shape[0]
+09:     cdef int Ny = camera_grid.shape[1]
+10:     cdef int Nz = camera_grid.shape[2]
11:
12:     cdef int i, j, k
13:
14:     cdef double val, r, g, b, a
15:
+16:     cdef np.ndarray image_np = np.zeros((Ny, Nz, 3))
+17:     cdef double[:, :, :] image = image_np
18:
19:     for i in range(Nx):
20:
21:         for j in range(Ny):
22:
23:             for k in range(Nz):
24:
+25:                 val = log(camera_grid[i, j, k] + 1e-9)
26:
27:                 r = exp(-((val-9.0)*(val-9.0))/1.0) \
28:                     + 0.1*exp(-((val-3.0)*(val-3.0))/0.1) \
+29:                     + 0.1*exp(-((val+3.0)*(val+3.0))/0.5)
30:
31:                 g = exp(-((val-9.0)*(val-9.0))/1.0) \
32:                     + exp(-((val-3.0)*(val-3.0))/0.1) \
+33:                     + 0.1*exp(-((val+3.0)*(val+3.0))/0.5)
34:
35:                 b = 0.1*exp(-((val-9.0)*(val-9.0))/1.0) \
36:                     + 0.1*exp(-((val-3.0)*(val-3.0))/0.1) \
+37:                     + exp(-((val+3.0)*(val+3.0))/0.5)
38:
39:                 a = 0.6*exp(-((val-9.0)*(val-9.0))/1.0) \
40:                     + 0.1*exp(-((val-3.0)*(val-3.0))/0.1) \
+41:                     + 0.1*exp(-((val+3.0)*(val+3.0))/0.5)
42:
+43:                 image[j, k, 0] = a*r + (1-a)*image[j, k, 0]
+44:                 image[j, k, 1] = a*g + (1-a)*image[j, k, 1]
+45:                 image[j, k, 2] = a*b + (1-a)*image[j, k, 2]
46:
+47:     return image_np

```

for dataslice in camera_grid:

ForEachslices $\in$ camera_grid

$$(r, g, b, \alpha) = T(\log(s + \epsilon))$$

$$I_R = \alpha r + (1 - \alpha)I_R$$

$$I_G = \alpha g + (1 - \alpha)I_G$$

$$I_B = \alpha b + (1 - \alpha)I_B$$

Figure 2: Cython Implementation

Since this loop is executed for every slice in the volume, Python interpreter overhead significantly affects performance. To optimize this section, the compositing routine was rewritten in Cython with static type declarations and compiled into a C extension module. This allows the loop to run as compiled C code instead of being interpreted by Python.

Performance benchmarking showed that the Python implementation required 0.4979 seconds, while the Cython version completed in 0.1146 seconds, resulting in a $4.34\times$ speedup. This improvement demonstrates that reducing interpreter overhead in the compositing loop significantly accelerates the rendering process.

The primary advantage of this technique is the reduction of interpreter overhead and improved efficiency of numerical operations. However, Cython requires additional compilation steps and explicit type declarations, which increases development complexity compared to pure Python implementation

2.3.2 Parallelization: Multiprocessing and Dask

To further improve performance, parallel computing techniques were introduced to distribute the workload across multiple processing units. Volume rendering is computationally intensive and involves operations that can be performed independently across different viewing angles or data slices. This makes the problem well-suited for parallel execution.

The first approach explored was Python's multiprocessing module. Multiprocessing allows the creation of multiple independent processes that can execute tasks concurrently across multiple CPU cores. In this implementation, the rendering tasks for different viewing angles were distributed among multiple worker processes. Each worker was responsible for computing a rendered image independently, enabling concurrent execution of the rendering pipeline.

However, multiprocessing introduces overhead due to process creation, inter-process communication, and data transfer between processes. This overhead can become significant when large datasets are involved, particularly when the data must be serialized and copied between processes.

To address these limitations, the Dask parallel computing framework was also evaluated. Dask provides a higher-level parallel execution model that supports task scheduling and efficient workload distribution. It constructs a task graph representing computational dependencies and executes tasks using a distributed or multi-threaded scheduler.

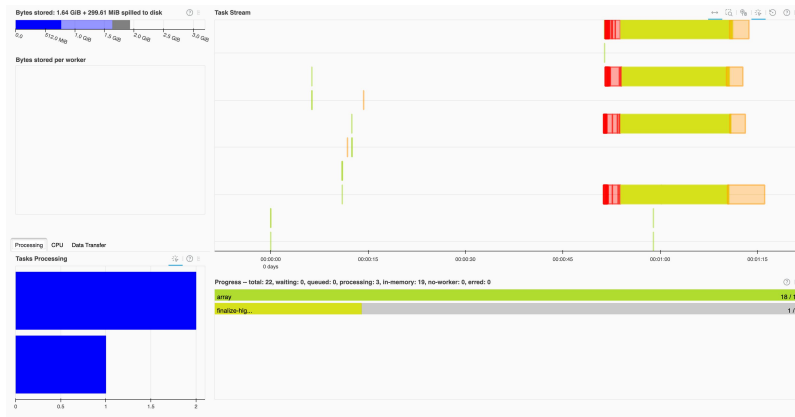


Figure 3: Dask Dashboard

1. Multiprocessing Implementation:

Multiprocessing was implemented using Python's multiprocessing.Pool to parallelize the rendering of different camera angles. Instead of rendering each view sequentially, the rendering function was executed concurrently across multiple processes. A pool of worker processes was created, and each worker was assigned a rendering task corresponding to a specific viewing angle.

The rendering function `render_view()` was mapped to a list of viewing angles using the process pool. Each process independently performed the interpolation, transfer function computation, and compositing steps for its assigned angle and produced a rendered image. Since each rendering task operates on the same input datacube but generates separate outputs, the tasks could be executed in parallel without dependencies.

However, multiprocessing introduced additional overhead due to process creation and inter-process communication. Large data structures such as the datacube had to be serialized and transferred to worker processes, which increased memory usage and reduced the overall performance gains.

2. Dask Implementation:

In addition to multiprocessing, the Dask parallel computing framework was evaluated to manage task parallelism more efficiently. Dask enables parallel execution by constructing a task graph that represents the dependencies between computational tasks. In this implementation, each rendering operation for a different viewing angle was defined as a delayed task using `dask.delayed()`.

The delayed rendering tasks were collected into a list and executed using the Dask scheduler. When computation was triggered, Dask distributed the tasks across available CPU cores and executed them concurrently. Compared to the multiprocessing approach, Dask provides more flexible scheduling and better scalability for larger workloads.

By parallelizing the rendering tasks across multiple CPU cores, both multiprocessing and Dask aimed to reduce the overall execution time of generating multiple rendered views of the datacube.

2.3.3 GPU Acceleration: PyTorch Implementation

The final optimisation technique explored was hardware acceleration utilising a GPU. GPUs are extensively used in high performance computing to speed up arithmetic-heavy computational workloads. Because volume rendering involves updating pixel values across an image grid independently, it is a problem that is highly suitable for the parallel computation cores available on a modern GPU.

To implement this acceleration, PyTorch was utilised. PyTorch is a static computation graph tensor library that provides a similar API to NumPy. By migrating the volumetric data structures from system RAM to GPU memory, PyTorch automatically compiles the numerical computations applied to that data into highly parallelised GPU code.

In our implementation, the entire volume rendering pipeline was adapted to operate on PyTorch tensors rather than standard Python or NumPy data types.

Grid Interpolation: the 3D camera grid transformation were replaced with PyTorch’s native `grid_sample`. This performs bilinear interpolation of the entire 3D camera grid simultaneously on the hardware accelerator.

```
1 # Interpolate the datacube onto the camera grid
2 datacube_t = datacube_t.unsqueeze(0).unsqueeze(0) # Shape: (1, 1, D, H, W)
3 camera_grid = F.grid_sample(datacube_t, grid, mode='bilinear', align_corners=True)
4 camera_grid = camera_grid.squeeze() # Shape: (N, N, N)
```

Listing 1: GPU-accelerated grid interpolation using PyTorch

Vectorised Transfer Function: The mathematical operations within the transfer function were rewritten using PyTorch equivalents (e.g., `torch.exp` instead of `np.exp`). This ensures that operations are performed on the entire input tensor at once.

Parallel Compositing: During the alpha blending stage, 2D slice matrices were processed using `torch.stack` and element-wise tensor multiplication to generate the final RGB channels.

```
1 # Initialize the image tensor
2 image = torch.zeros((N, N, 3), device=device)
3
4 for i in range(N):
5     a_slice = a[i].unsqueeze(-1) # Shape: (N, N, 1)
6     rgb_slice = torch.stack((r[i], g[i], b[i]), dim=-1) # Shape: (N, N, 3)
7     image = a_slice * rgb_slice + (1 - a_slice) * image # Alpha blending
```

Listing 2: Parallelized alpha blending compositing loop

3 Experimental Setup

3.1 Setup for profiling the baseline

All baseline profiling was done on the hardware and software listed in the tables 1 and 2.

4 Results

4.1 Validation of Algorithm

To verify the correctness of the applied optimisations, the rendered output images from the Multiprocessing, Dask, Cython, and GPU implementations were both visually and numerically compared against the baseline serial Python execution. As illustrated in Figure 4, all optimised versions successfully produced identical 3D volume renderings. Furthermore, a pixel-wise residual analysis yielded a negligible Mean Absolute Error (MAE), confirming that the mathematical and structural integrity of the algorithm was strictly preserved across all parallelisation and compiler techniques.

Table 1: Hardware Specifications	
Machine	Apple MacBook Pro
Processor	Apple M1 Pro (8-core CPU, 14-core GPU)
Memory	16 GB RAM

Table 2: Software and Environment				
Operating System	macOS	Tahoe	(Version 26.3.1)	
Runtime	Python 3.14.2			
Profiling Tools	cProfile, line_profiler, memory_profiler			

Table 4: Software Environment for Performance Optimisations					
Operating System	macOS	Tahoe	(Version 26.3.1)		
Runtime	Python 3.11.2				
Cython Tools	Cython compiler, setup-tools, gcc/clang				
Dask Tools	Dask distributed scheduler, Dask dashboard				
Multiprocessing Tools	Python multiprocessing.Pool				
GPU Tools	PyTorch 2.12.0 with 'mps' backend				

Table 3: Hardware Specifications for Performance Optimisations	
Machine	Apple MacBook Air
Processor	Apple Silicon M4 (10-core CPU)
GPU	10-core Integrated GPU
Memory	16 GB Unified Memory

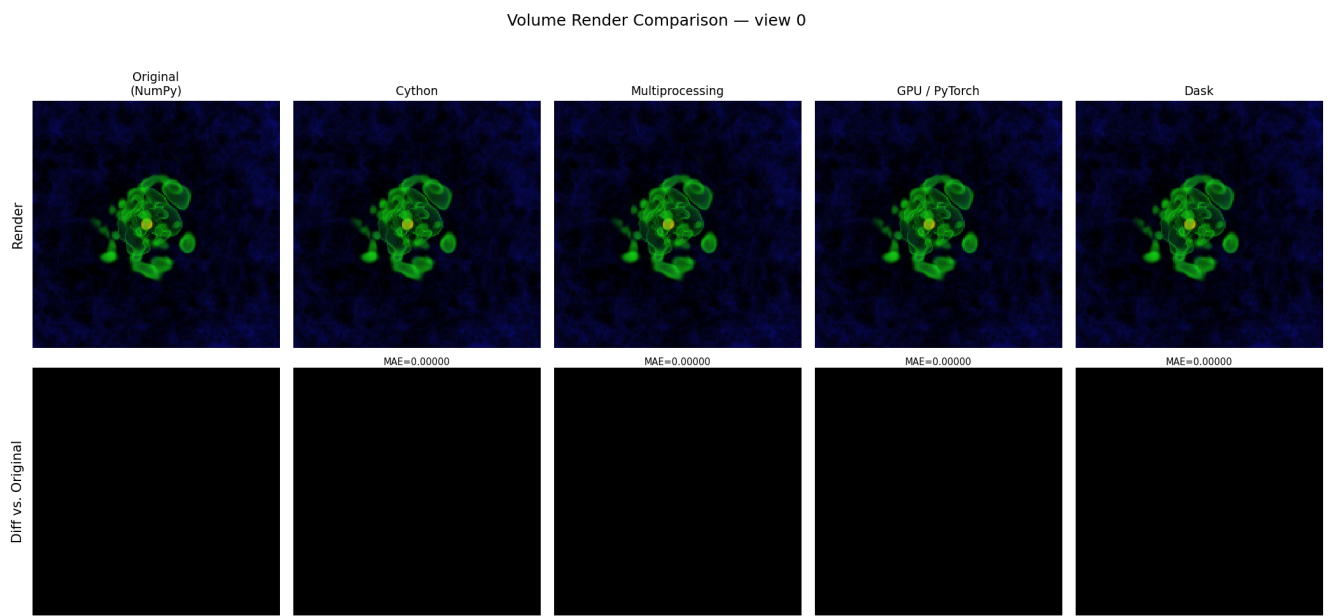


Figure 4: **Validation of optimisation correctness.** The top row displays the rendered 3D volume output for viewing angle 0 across the baseline and all four optimised implementations, demonstrating identical visual structures. The bottom row visualises the pixel-wise absolute difference (residuals) between each optimised baseline. The corresponding Mean Absolute Error (MAE) is also annotated for each method. The zero MAE and zero residual values confirm that all optimisation techniques successfully preserve the integrity of the original algorithm.

4.2 Performance Characterisation (Baseline)

4.2.1 Profiling Setup

The performance of the baseline implementation was analyzed using the profiling methods described in Section 2.2. Function-level profiling was performed using “cProfile” in order to identify the most time-consuming functions. These functions were then analyzed in more detail using the “line profiler”. In addition, the memory profiler was used to observe the memory consumption during program execution.

All measurements were performed with plotting disabled to avoid influencing the execution time.

4.2.2 Function-Level Profiling with “cProfile”

The baseline implementation was first analyzed using the “cProfile” profiler.

Table 5 shows the functions with the highest cumulative execution time.

Table 5: Functions with the highest cumulative execution time according to cProfile			
Function	Calls	Cum. Time [s]	Time per Call [s]
interp _n ()	10	6.830	0.683
_evaluate_linear()	10	4.371	0.437
transferFunction()	1800	2.634	0.001
_find_indices()	10	2.136	0.214

It can be observed that a large portion of the runtime is spent inside the interpolation routine `interpn()` from the SciPy library. This function internally calls `_evaluate_linear()`, which also contributes significantly to the total execution time.

In addition, the function `transferFunction()` accounts for a noticeable share of the runtime due to its high number of calls (1800). Therefore, both the interpolation routine and the transfer function were further analyzed using the line profiler.

4.2.3 Line-Level Profiling with “line profiler”

The function `transferFunction()` was analyzed using the line profiler in order to determine which specific operations contribute most to the execution time.

The analysis shows that the function was executed 1800 times during the rendering process and required a total runtime of approximately 3.07 seconds.

Table 6 shows the most time-consuming operations within the function.

Table 6: Most time-consuming operations in `transferFunction()` according to the line profiler

Operation	Hits	Time [ms]	Percentage
Exponential evaluation for red channel	1800	728	23.7
Exponential evaluation for green channel	1800	739	24.1
Exponential evaluation for blue channel	1800	844	27.5
Exponential evaluation for alpha channel	1800	756	24.6

Each colour channel requires approximately the same execution time, since the implemented operations are nearly identical. Therefore, only the red channel is analyzed in more detail as a representative example. The execution time of the individual lines is shown in Table 7.

Table 7: line profiler results for the red channel in `transferFunction()`

Line	Hits	Time [ms]	Percentage
<code>np.exp(-(x - 9.0) ** 2) / 1.0)</code>	5400	267.9	8.7
<code>np.exp(-(x - 3.0) ** 2) / 0.1)</code>	1800	226.9	7.4
<code>np.exp(-(x + 3.0) ** 2) / 0.5)</code>	1800	233.8	7.6

The function `transferFunction()` is executed 1800 times during the rendering process. However, some lines in the profiler results show 5400 executions. This is due to the way how Python traces the number of hits of each line and for each addition a new hit is counted. Therefore this can be ignored because those are not redundant function calls.

The results indicate that the majority of the runtime is spent in the evaluation of exponential functions used to compute the Gaussian components of the transfer function.

Each color channel (red, green, blue, and alpha) consists of three Gaussian functions, resulting in a large number of exponential evaluations. Since the function is executed many times during rendering, these repeated computations contribute significantly to the overall runtime.

Therefore, the evaluation of the Gaussian functions represents the main computational bottleneck within the transfer function.

4.2.4 Memory Profiling

In addition to the execution time, the memory consumption of the baseline implementation was analyzed using the memory profiler. The goal of this analysis was to identify operations that require large memory allocations and to verify that no unintended memory growth occurs during execution.

Table 8 summarizes the most significant memory allocations observed during the execution of the program.

Table 8: Major memory allocations identified using the memory profiler

Operation	Memory Increase	Description
Loading datacube	63.3 MiB	Conversion of HDF5 dataset to NumPy array
Creation of mesh grid	132.2 MiB	Generation of query grid using <code>meshgrid</code>
Interpolation step	734.2 MiB	Creation of interpolated camera grid

The largest increase in memory usage occurs during the interpolation step, where the function `interp()` generates the three-dimensional camera grid used for volume rendering. This operation allocates approximately 734 MiB of additional memory.

Another notable memory allocation occurs during the generation of the query grid using `numpy.meshgrid()`, which creates three large three-dimensional arrays.

The total maximum memory usage during execution reaches approximately 1.3 GiB. However, the memory consumption remains stable after the initial allocations, indicating that there are no memory leaks in the implementation.

4.3 Optimisation Results

To evaluate the effectiveness of the implemented optimizations, performance measurements were conducted for the Cython optimization, the parallelization approaches using multiprocessing and Dask and using PyTorch GPU Acceleration. The execution time of each implementation was measured and compared against the baseline Python implementation. Evaluation was carried out with the hardware and software environment detailed in tables 3 and 4

1. Cython Performance Comparison

The performance of the Cython implementation was compared with the original Python implementation for the compositing stage of the volume rendering pipeline. The baseline Python version required 0.4865 seconds, while the Cython implementation completed the same operation in 0.1167 seconds, resulting in a 4.2× speedup as mentioned in figure 5. This improvement is primarily due to the reduction of Python interpreter overhead, as the computational loop was compiled into optimized C code.

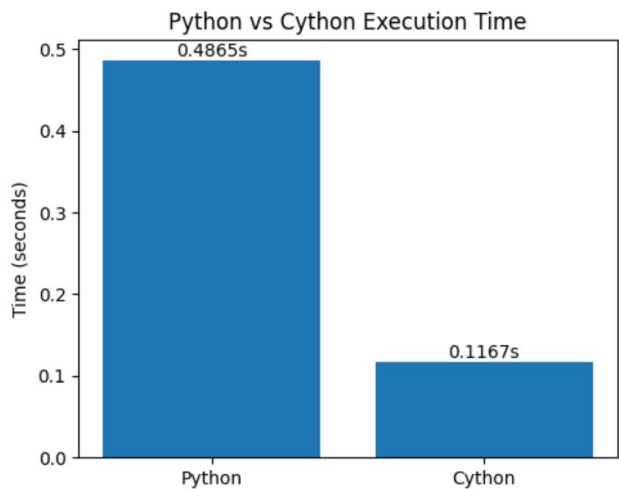


Figure 5: Python vs Cython Execution Time

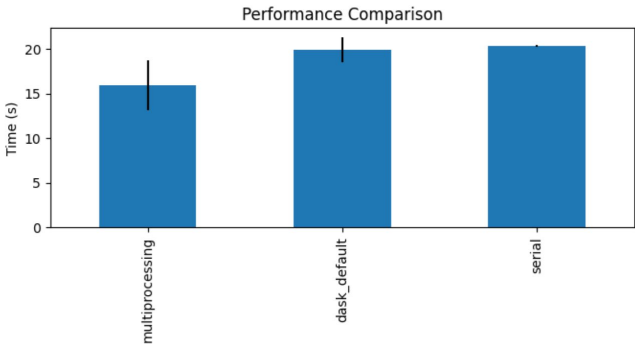


Figure 6: Parallelization Performance Comparison

2. Parallelization Performance Comparison

Parallelization was evaluated using multiprocessing and Dask, and compared with the serial baseline implementation. The multiprocessing approach achieved the best performance with an execution time of approximately 16 seconds, benefiting from parallel execution across multiple CPU cores. You can see the results in Figure 6. The Dask implementation required around 20 seconds, performing similarly to the serial execution time of about 20.5 seconds. The additional scheduling overhead in Dask likely limited its performance gains for this workload.

3. GPU Performance Comparison

The PyTorch GPU implementation yielded the most dramatic performance improvements of all tested optimisations. Figure 7 illustrates the execution time scaling of the Original (NumPy) CPU baseline against the GPU implementation. This represents a roughly 4.5× speedup strictly within the mathematical rendering kernel. Because the transfer function consists of numerous exponential operations applied across millions of data points, it is highly compute-bound. The CPU must process these element-wise operations sequentially, whereas the PyTorch tensors evaluate them massively in parallel, fully leveraging the GPU’s Single Instruction, Multiple Data (SIMD) architecture.

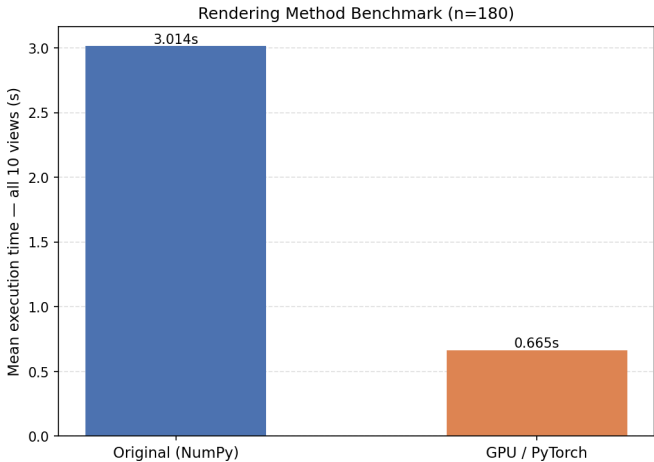


Figure 7: Original NumPy vs GPU/PyTorch

Overall, the results show that Cython significantly accelerates compute-intensive loops, while multiprocessing provides effective parallel speedup for independent rendering tasks, but hardware acceleration via the GPU provides the most viable scalability for high-resolution datasets.

4.4 Scalability ablation

To further characterise the computational bounds of the implementations, a scalability ablation study was performed by varying the camera grid resolution n from 64^3 to 240^3 voxels. Increasing n exponentially scales the mathematical workload of the 3D grid interpolation and compositing algorithms. This benchmark was performed on a 1.14GB volume to stress the system’s memory architecture.

The ablation study evaluated both the "Native" end-to-end pipelines (including grid generation, interpolation, and rendering) and "Kernel-only" execution (isolating the rendering loop from the grid pre-computation).

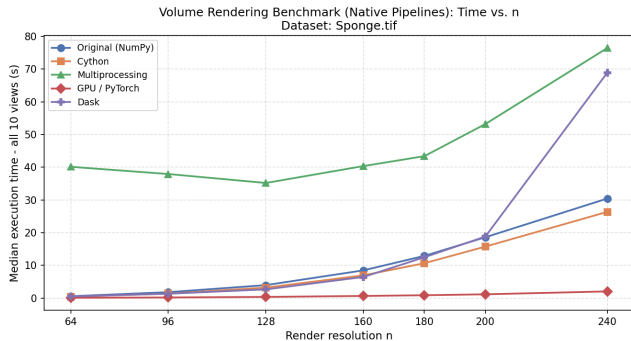


Figure 8: Native end-to-end execution time vs. Grid Resolution (n).

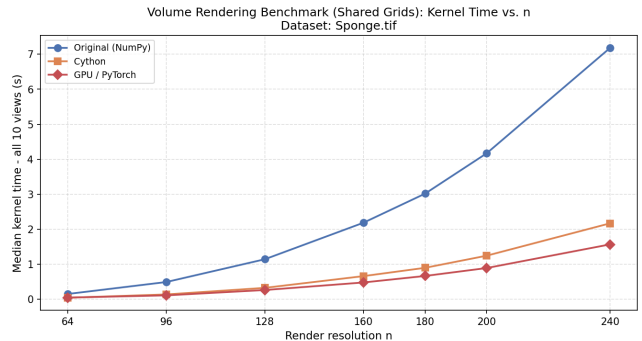


Figure 9: Kernel-only execution time vs. Grid Resolution (n).

As illustrated in the Native pipeline benchmark (Figure 8), the original NumPy and Cython implementations exhibit steep cubic scaling curves. At the highest tested resolution ($n = 240$), the baseline execution time degrades to 30.41 seconds (median). The Dask implementation struggles similarly at high resolutions, scaling up to 68.89 seconds, largely due to the overhead of managing task graphs for increasingly large intermediate memory allocations.

The most prominent anomaly observed is the Multiprocessing implementation. Regardless of the low computational workload at $n = 64$, the execution time consistently hovers around 40 seconds, scaling up to 76.42 seconds at $n = 240$. This is a textbook demonstration of the inter-process communication bottleneck. Because the `Sponge.tif` dataset is over 1.1 GB in size, serialising and copying this massive memory block across isolated worker processes completely saturates the system's RAM bandwidth. The cost of data movement vastly outweighs the benefits of CPU parallelism.

Conversely, the GPU implementation exhibits a shallow scaling curve across both plots. At $n = 240$, the native GPU pipeline only requires 2.019 seconds. Furthermore, the Kernel-only profiling (Figure 9) revealed that the raw grid generation and interpolation using PyTorch took only 1.566 seconds compared to SciPy's 7.178 seconds. This confirms that hardware accelerators highly optimise dense memory accesses, effectively mitigating the cubic scaling penalty associated with high-resolution volume rendering.

5 Discussion

In the performance analysis section, the impact of different optimisation strategies on the efficiency of the volume rendering pipeline is shown. It was found that a major part of the execution was being taken by the interpolation routine along with the transfer function during profiling of the baseline implementation. This was a major contributor for repeated calculations involving exponentials.

The implementation involving the Cython programming language was extremely beneficial for the efficiency of the compositing stage. This was because the Python interpreter was minimised. The execution time was reduced by more than four times compared to the original Python implementation, as the critical loop was compiled to C code with the use of static typing.

For the parallelisation strategies, the results were mixed. The implementation using multiprocessing was faster, as the execution time was reduced by using more than one core for the execution of the code. However, the results were not as good for large datasets, as the datacube was copied, leading to high memory bandwidth use. The implementation using the Dask library showed similar or slower execution compared to the baseline, as the overhead of the schedule as well as the management of the graph was high. The GPU-based implementation using PyTorch showed the highest performance gain. This is because the GPU was used for acceleration to achieve maximum parallelization of tensor operations like interpolation and exponential calculation, which are normally used in the transfer function. Therefore, the GPU-based implementation provided the highest level of scalability and the lowest execution times for the code. The GPU-based implementation performed better for higher resolutions of the grid.

The results have shown that although CPU-based optimisations can provide some benefits such as reduced interpreter overheads and parallelization, the best solution is still to use a GPU.

6 Conclusion

In this project, a Python-based volume rendering pipeline was tested along with different methods for optimising it for better efficiency. From the profiling results, it was found that the major performance bottleneck was in the interpolation and calculation of the transfer function.

In order to optimise the volume rendering pipeline for better efficiency, different methods like compiler optimisation using Cython, CPU parallelisation using multiprocessing and Dask libraries, and GPU acceleration using PyTorch were tested. From the results of the experiment, it was found that GPU acceleration provided better results compared to other methods in terms of improving the efficiency of the volume rendering pipeline. In fact, when different computationally intensive operations are executed in parallel on a GPU that has different cores, GPU acceleration using PyTorch provides better results compared to other methods. This is because GPU acceleration efficiently executed different computationally intensive operations compared to other methods.

It can be identified that overall hardware acceleration yields the best results in terms of improving the performance of volume rendering applications compared to other techniques such as compiler optimization using Cython and CPU parallelization using multiprocessing and Dask.

References

- [1] Philip Mocz. *Create Your Own Volume Rendering (With Python)* — *medium.com*. <https://medium.com/swlh/create-your-own-volume-rendering-with-python-655ca839b097>. [Accessed 19-02-2026].
- [2] pmocz. *GitHub - pmocz/volumerender-python: Volume Render a Datacube* — *github.com*. <https://github.com/pmocz/volumerender-python>. [Accessed 19-02-2026].
- [3] Quantitative Imaging Facility (QIM). *QIM Data Repository*. <https://qim.compute.dtu.dk/data-repository/>. [Accessed: 24-02-2026]. Technical University of Denmark (DTU).